

CS-112

Object Oriented Programming

DEPARTMENT OF COMPUTER SCIENCE

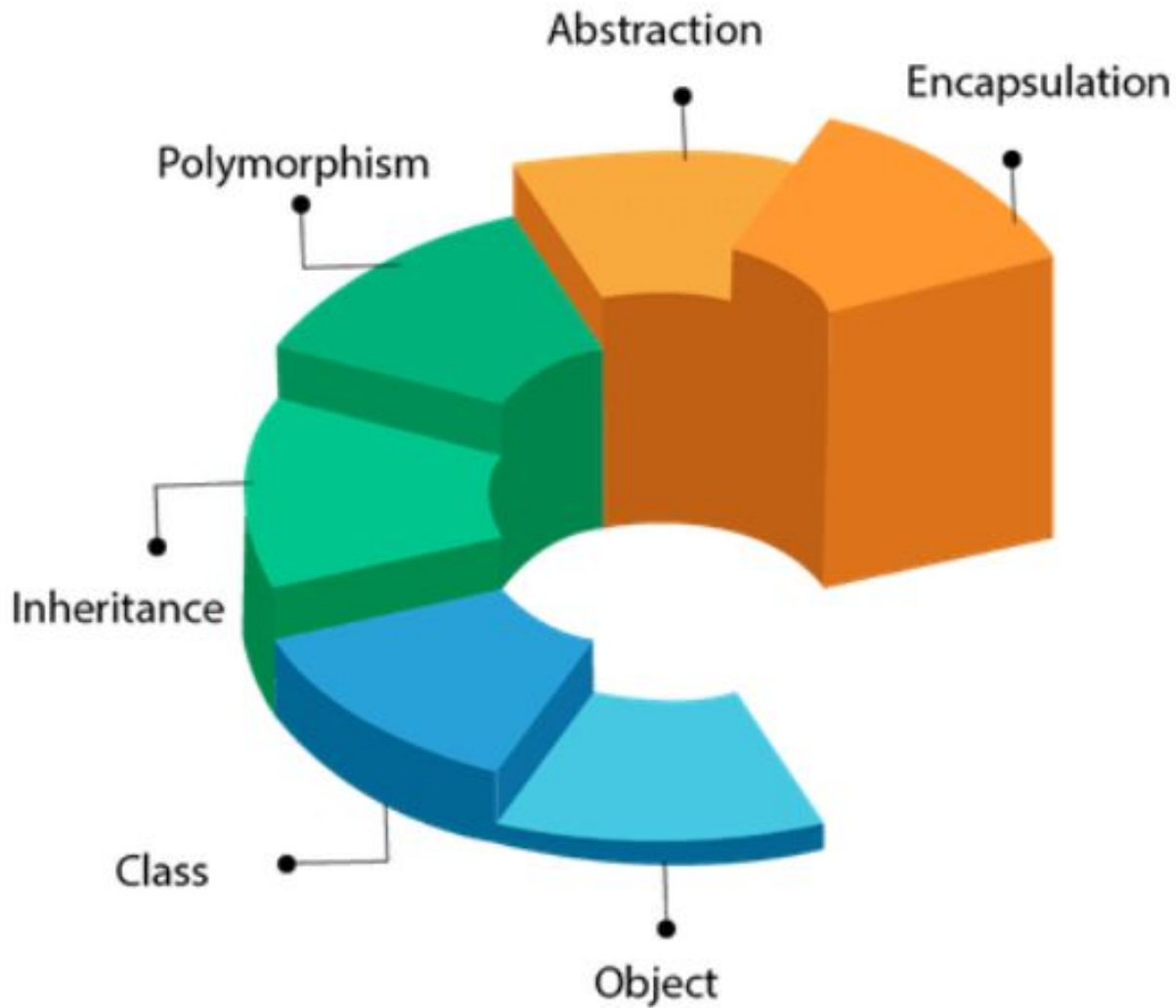
COURSE INSTRUCTOR : MS. HABIBA ARSHAD

Topics To be Covered Today

- ❑ OOP principles (abstraction, encapsulation, inheritance and polymorphism)

CLO Covered

CLO1: Understand the underlying concepts of object-oriented paradigm.



Object-Oriented Programming

Object-Oriented Programming is a methodology or paradigm to design a program using classes and objects.

It simplifies software development and maintenance by providing some concepts:

Cont...

Apart from these concepts, there are some other terms which are used in Object-Oriented design:

- ❑ Coupling
- ❑ Cohesion
- ❑ Association
- ❑ Aggregation
- ❑ Composition

Abstraction

Abstraction

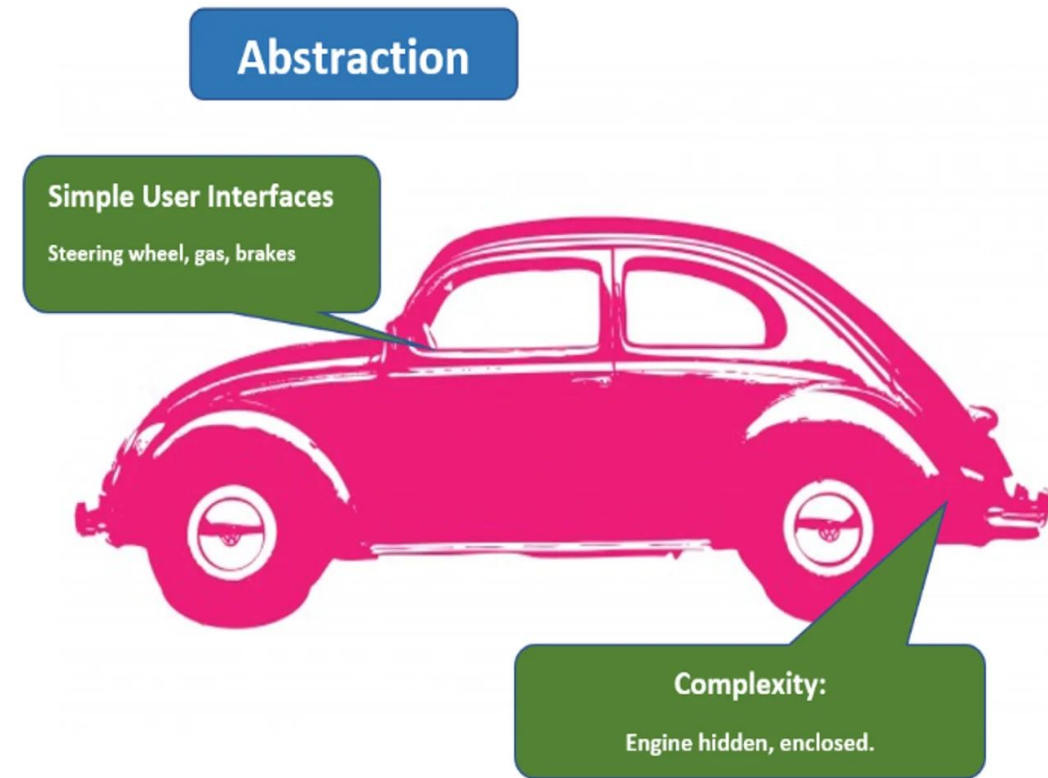
Abstraction is a way to cope with complexity.

Principle of abstraction:

“Capture only those details about an object that are relevant to current perspective”

For Example

- For example, you don't have to know all the details of how the engine works to drive a car.
- A driver only uses a small selection of tools: like gas pedal, brake, steering wheel, blinker. The engineering is hidden from the driver. To make a car work, a lot of pieces have to work under the hood, but exposing that information to the driver would be a dangerous distraction.



Benefits of Abstraction

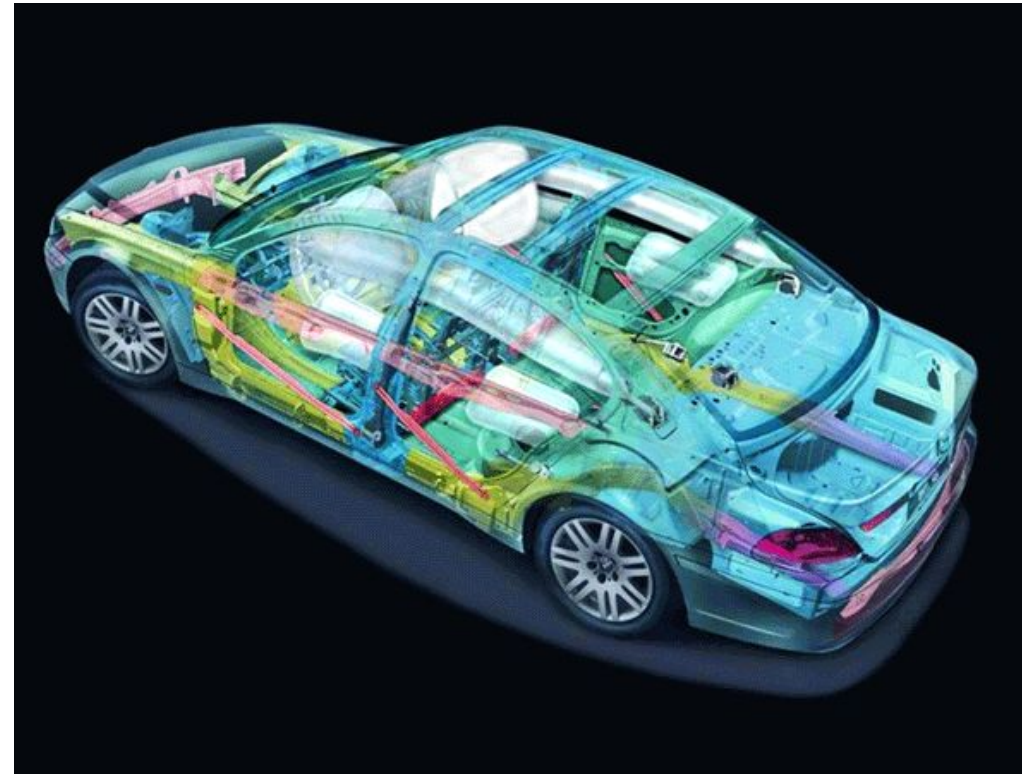
- Simple, high level user interfaces
- Complex code is hidden
- Security
- Easier software maintenance
- Code updates rarely change abstraction

Example – Abstraction

DRIVER'S VIEW

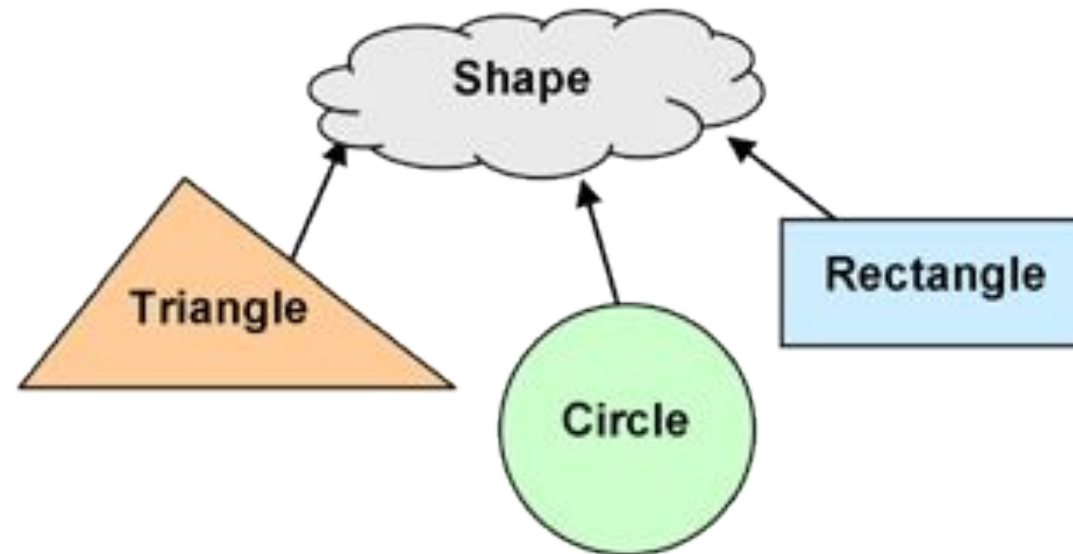


ENGINEER'S VIEW

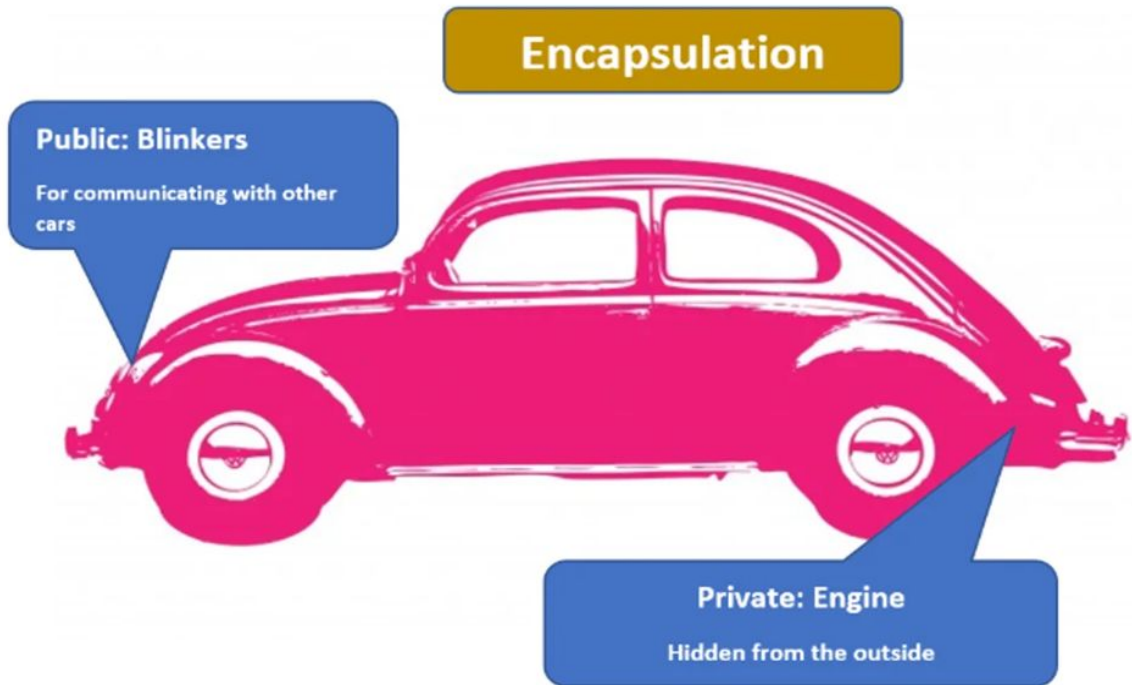


Abstraction

- Abstraction hides details at the design level, while Encapsulation hides details at the implementation level.
- The abstract classes and interfaces are used to achieve abstraction in Java.



Encapsulation



Encapsulation

“Encapsulation means containing all important information inside an object, and only exposing selected information to the outside world.”

Example – Encapsulation

Ali stores his personal information and knows how to translate it to the desired language

We don't know

- How the data is stored
- How Ali translates this information

Encapsulation

Encapsulation provides abstraction at implementation level

Separates external view (behavior) from internal view (state)

Encapsulation protects integrity of an object's data

Access modifier keywords are used for encapsulation in object oriented programming.

- For example, encapsulation in java is achieved using private, protected and public keywords.

Provide setters and getters (accessors / mutators) to allow controlled access to private data

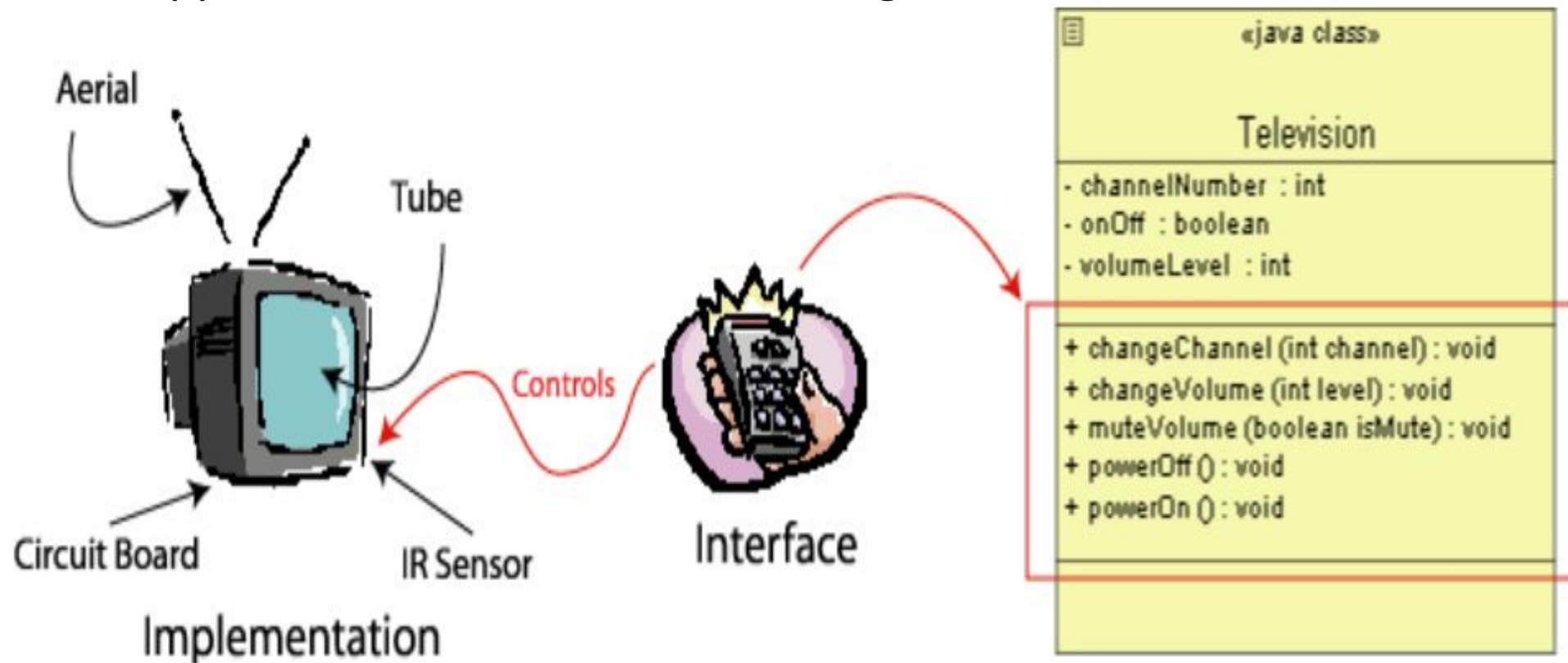
- The **getter** method returns the variable value, and the **setter** method sets the value.

Encapsulation

Provides abstraction between an object and its clients.

Protects an object from unwanted access by its clients.

A Bank application forbids a client to change an account's balance



Encapsulation

Encapsulation is a process of binding the data members (attributes) and methods together.

The encapsulation restricts direct access to important data.

The best example of the encapsulation concept is making a class where the data members are private and methods are public to access through an object. In this case, only methods can access those private data.

Interface



Interface

- We learned that object encapsulates data and behavior
- So how objects interact with each other?
 - Each object provides an interface (operations)
 - Other objects communicate through this interface

Example – Interface of a Car

Steer Wheels

Accelerate

Change Gear

Apply Brakes

Turn Lights On/Off

Example – Interface of a Phone

Input Number

Place Call

Disconnect Call

Add number to address book

Remove number

Update number

Implementation

Provides services offered by the object interface

This includes

- Data structures to hold object attributes
- Functionality that provides required services

Example – Implementation of Address Book in a Phone

Data Structure

- SIM card

Functionality

- Read/write circuitry

Separation of Interface & Implementation

Means change in implementation does not effect object interface

This is achieved via principles of information hiding and encapsulation

Example – Separation of Interface & Implementation

A driver can drive a car independent of engine type (petrol, diesel)

A driver can apply brakes independent of brakes type (simple, disk)

Because interface does not change with the implementation

Advantages of Separation

Users need not to worry about a change until the interface is same

Low Complexity

Direct access to information structure of an object can produce errors

Messages

Messages

Objects communicate through messages

They send messages (stimuli) by invoking appropriate operations on the target object

The number and kind of messages that can be sent to an object depends upon its interface

Examples – Messages

A Person sends message (stimulus) “stop” to a Car by applying brakes

A Person sends message “place call” to a Phone by pressing appropriate button

Class & Methods

Class

The class is the essential Java construct. A class is a template or blueprint for objects. “collection of objects with same properties and functions”. To program in Java, you must understand classes and be able to write and use them.

Methods

a collection of statements that performs a sequence of operations to display a message on the console. It can be used even without fully understanding the details of how it works. It is used by invoking a statement with a string argument.

Classes

In an OO model, some of the objects exhibit identical characteristics (information structure and behaviour)

We say that they belong to the same class

Example – Class

Ali studies mathematics

Anam studies physics

Sohail studies chemistry

□ Each one is a Student

□ We say these objects are instances of the Student class

Example – Class

Ahsan teaches mathematics

Aamir teaches computer science

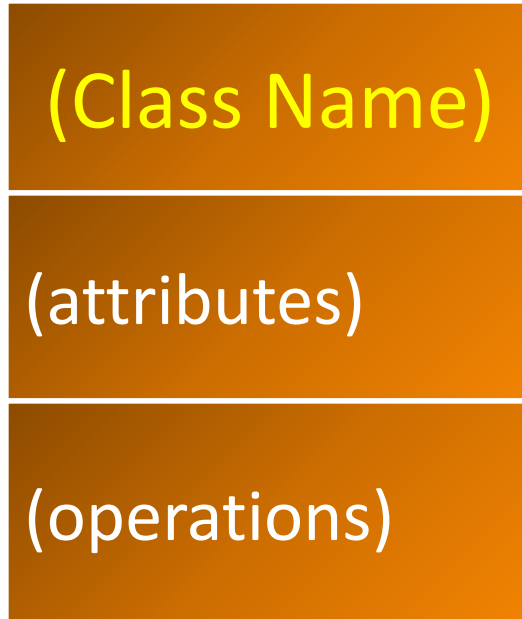
Atif teaches physics

□ Each one is a teacher

□ We say these objects are instances of the Teacher class

Graphical Representation of Classes

Normal Form

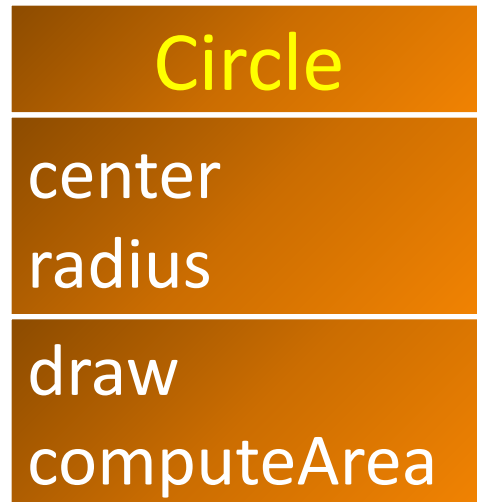


Suppressed Form



Example – Graphical Representation of Classes

Normal Form



Suppressed Form



Example – Graphical Representation of Classes

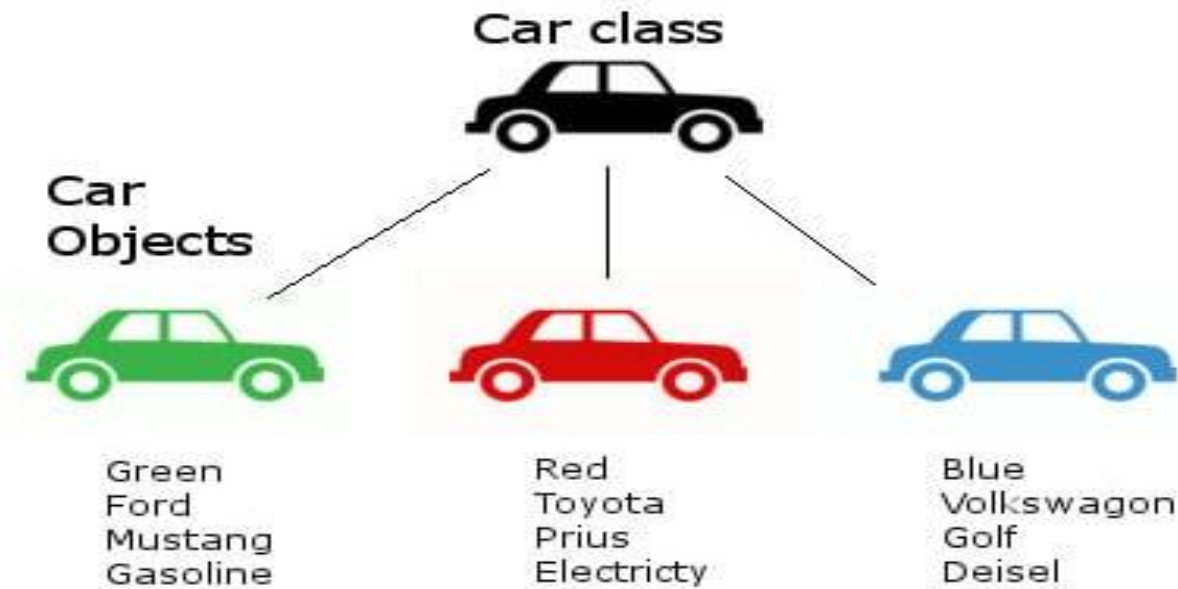
Normal Form

<i>Person</i>
name
age
gender
eat
walk

Suppressed Form

<i>Person</i>

Class Example



Car car1=new car();

Car car2=new car();

Car car3=new car();

Java Code illustration:Example

```
//Java Program to illustrate how to define a class and fields
//Defining a Student class.
class Student{
    //defining fields
    int id;//field or data member or instance variable
    String name;
    //creating main method inside the Student class
    public static void main(String args[]){
        //Creating an object or instance
        Student s1=new Student();//creating an object of Student
        //Printing values of the object
        System.out.println(s1.id);//accessing member through reference variable
        System.out.println(s1.name);
    }
}
```

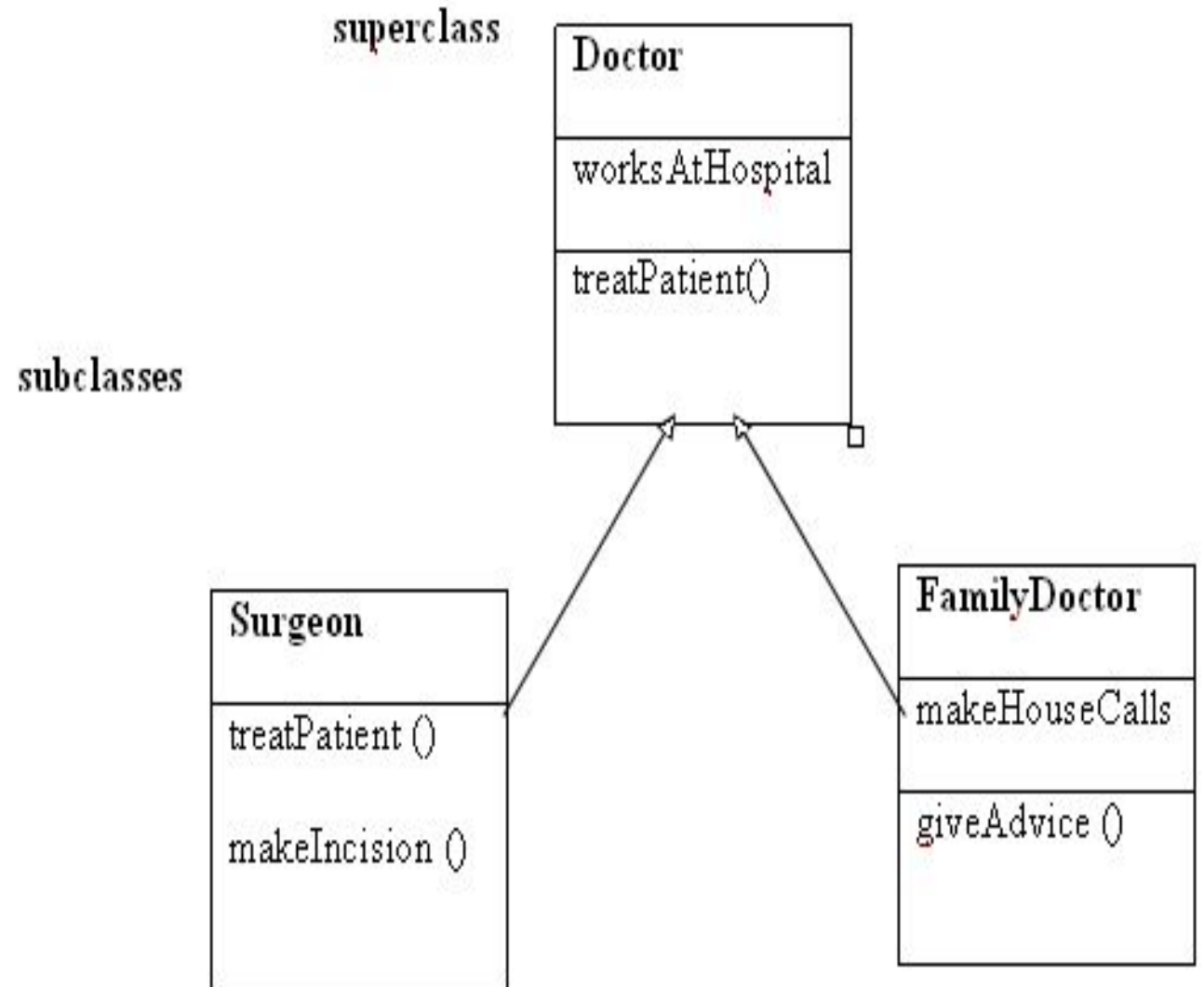
Inheritance

When an object is based on another object.

Inheritance is the mechanism of code reuse.

- Superclass :The object that is getting inherited
- Subclass : the object that inherits the superclass

We use extends keyword in java to implement inheritance.



Inheritance

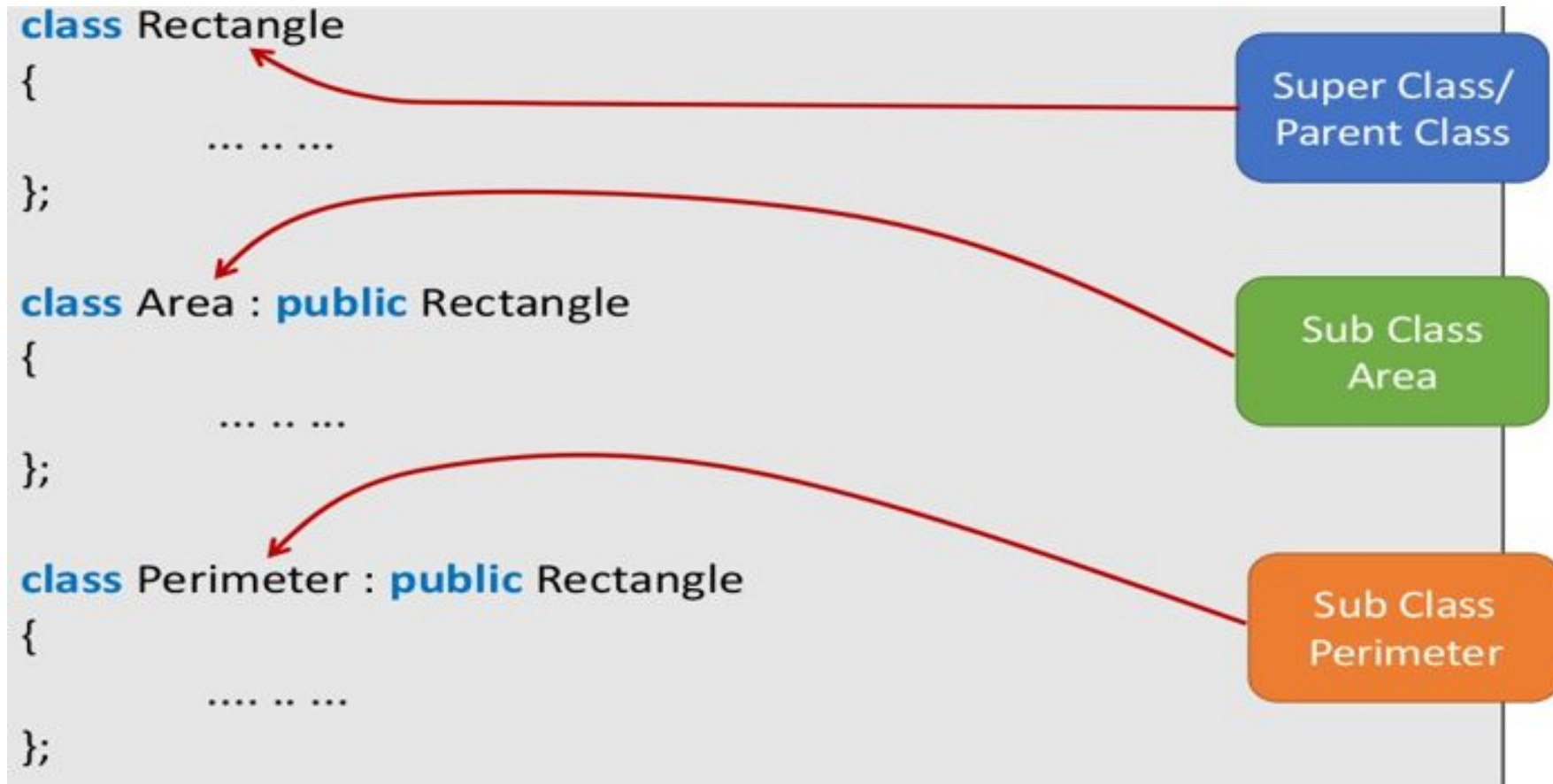
- The process of obtaining the data members and methods from one class to another class is known as inheritance.
- Inheritance in Object Oriented Programming can be described as a process of creating new classes from the existing classes.
- The class whose properties are inherited by other class is called the parent or base class where as the class which inherits properties of other class is called child or derived class.
- Inheritance makes the code reusable.

Inheritance Example

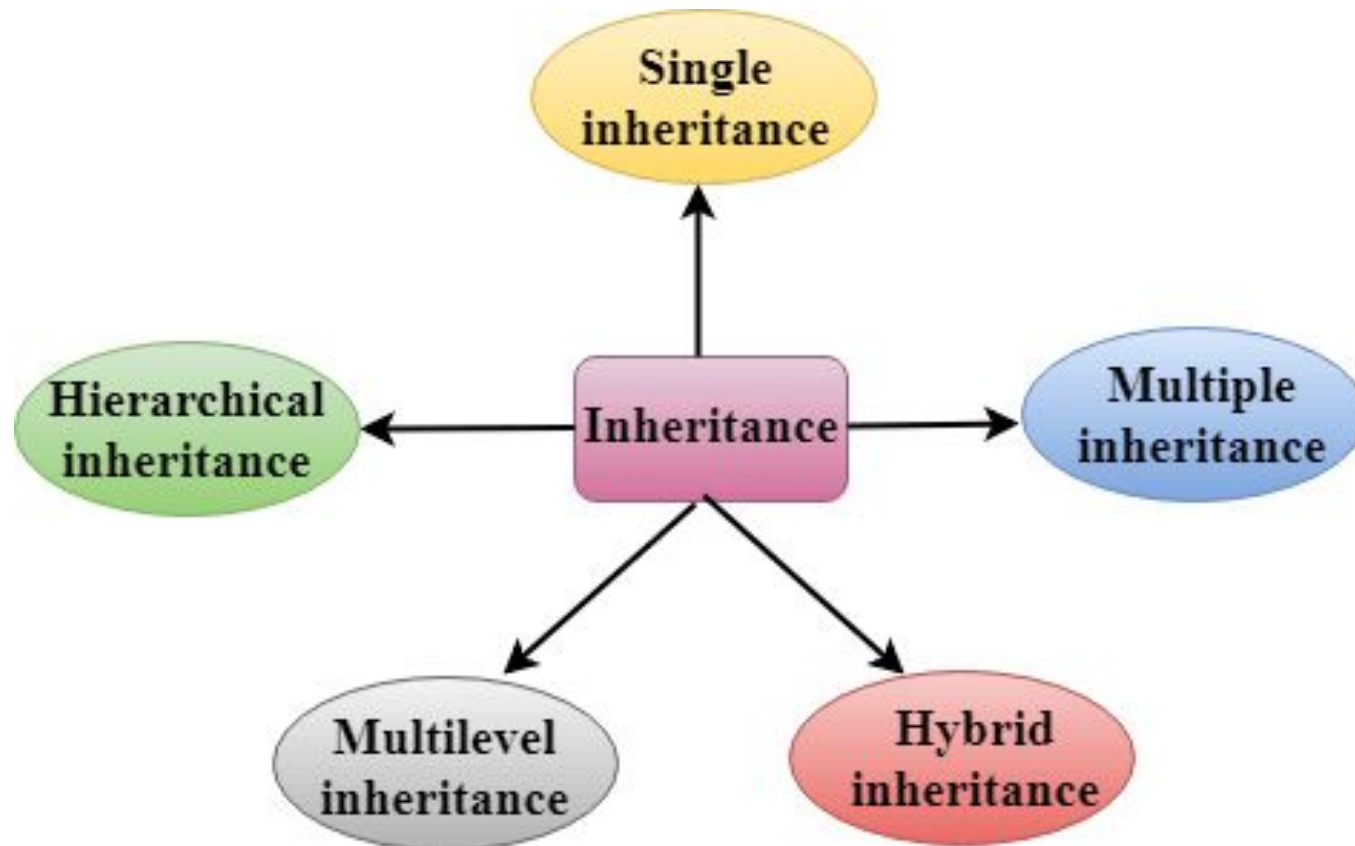
Real life Example of Inheritance



Inheritance Example



Types of Inheritance

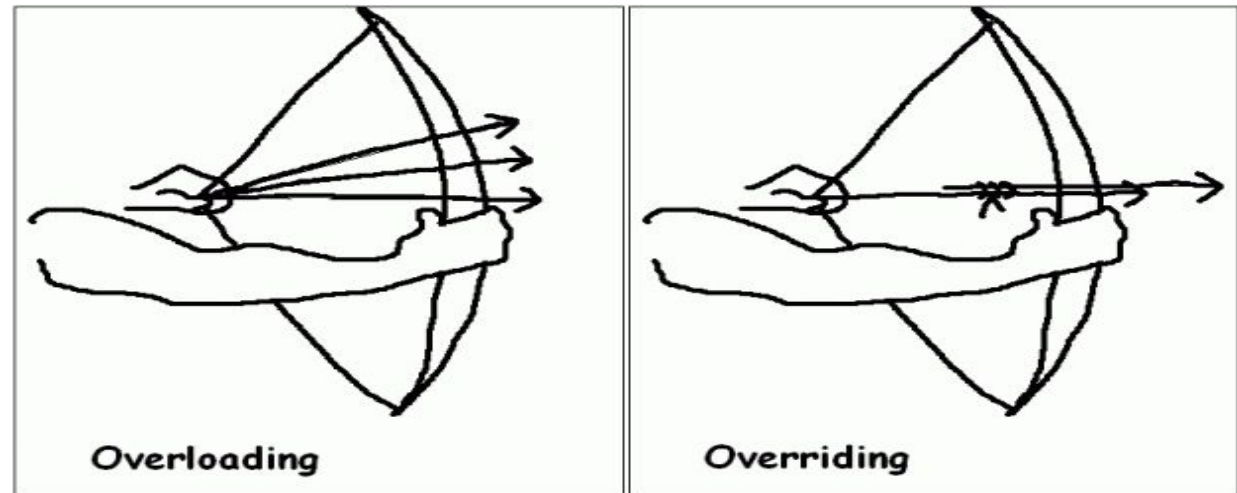


Polymorphism

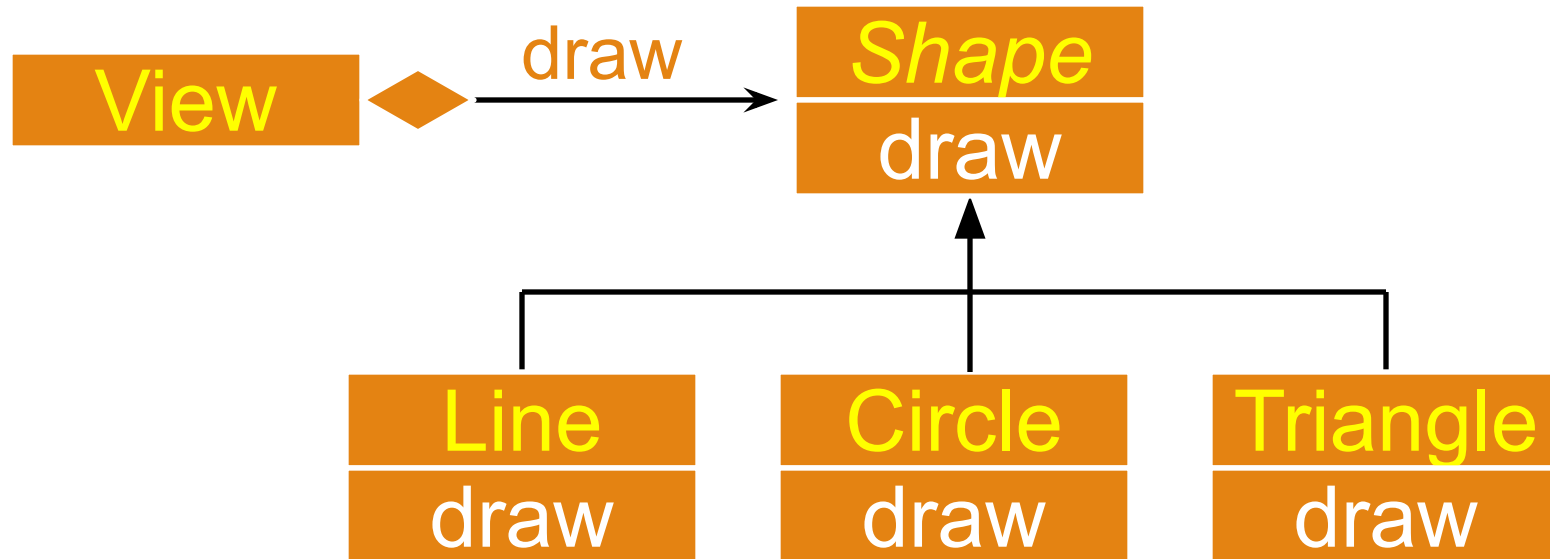
Concept where an object behaves differently in different situations.

Two types of polymorphism

- compile time polymorphism (overloading)
- runtime polymorphism (method overriding)



Example – Polymorphism



Example – Polymorphism

